

实验报告

课程名称：人工智能

姓名：马潇函 学号：2353912

2025 年 6 月 16 日

摘要

本文针对 Stable Diffusion 模型在特定艺术风格迁移中的应用需求，系统研究了多种微调策略的有效性和实用性。研究基于中国画风格数据集，对比分析了 SDv1.5 全参数微调、SDXL 部分层微调、LoRA 高效微调以及 DreamBooth 小样本微调等四种主要方法。实验结果表明，SDXL 架构相比 SDv1.5 在生成质量和细节表现方面具有显著优势，显著优于传统全参数微调方法。DreamBooth 方法展现了出色的小样本学习能力，仅用 7 张样本图像即可实现高质量的个性化风格迁移。此外，本研究还深入分析了各方法在跨语言理解、风格融合以及计算资源消耗等方面的特性，为扩散模型在实际应用中的方法选择提供了系统性的参考依据。研究过程中积累的工程实践经验，包括显存优化、数值稳定性保障等关键技术，对推动生成式 AI 在艺术创作领域的产业化应用具有重要价值。

关键词：Stable Diffusion；LoRA；部分层微调；DreamBooth；艺术风格迁移

目录

1	问题概述	4
1.1	研究背景	4
1.2	核心问题定义	4
1.3	技术路线与方法选择	4
1.4	实施策略与技术考量	5
1.5	进阶方向探索	5
2	算法设计	5
2.1	整体架构设计	5
2.2	数据预处理设计	6
2.3	核心训练算法	7
2.4	参数冻结策略	8
2.5	损失函数设计	9
2.6	优化策略设计	9
2.7	训练稳定性保障	10
3	算法实现	10
3.1	基础项算法实现	10
3.2	进阶项算法实现：SDXL 部分层微调	12
3.3	进阶项算法实现：LoRA 微调	14
3.4	进阶项算法实现：DreamBooth 微调	16
3.5	优化策略与工程实践	19
4	实验结果	20
4.1	基础模型与微调效果对比	20
4.2	不同数据集风格迁移实验	22
4.3	SDXL 模型及进阶微调方法效果	24
4.4	DreamBooth 小样本个性化实验	25
5	总结与分析	26
5.1	实验效果对比分析	26
5.2	资源效率与实用性评估	26
5.3	技术挑战与解决策略	26
5.4	方法适用性与选择建议	27
5.5	研究价值与未来展望	27
	参考文献	28

1 问题概述

1.1 研究背景

近年来，Stable Diffusion 等扩散模型在图像生成领域取得了突破性进展，这类模型通过学习数据分布的逐步去噪过程，能够从随机噪声中生成高质量的图像。不过，预训练的通用模型虽然具备强大的生成能力，但在面对特定艺术风格或领域需求时，往往难以精确地满足用户的期待。比如，当我们希望生成具有中国画特色的作品时，原始模型可能只能产生一些表面相似但缺乏深层文化内涵的图像。

这个问题的根源在于，预训练模型是在大规模通用数据集上训练的，其学到的是一种“平均化”的视觉表示，而特定艺术风格往往具有独特的色彩搭配、笔触特征和构图理念。因此，如何让这些强大的基础模型更好地适应特定风格需求，就成了一个亟待解决的技术问题。微调技术为这个挑战提供了可行的解决路径，通过在目标数据集上进行有针对性的训练，我们可以让模型“学会”特定的艺术表达方式。

1.2 核心问题定义

本次实验的核心任务是探索如何有效地对 Stable Diffusion 模型进行微调，使其能够生成符合中国画风格的高质量图像。这个问题看似简单，实际上涉及多个层面的技术挑战。首先是如何在保持模型原有泛化能力的同时，让它学会新的风格特征。如果微调过度，模型可能会过拟合到训练数据上，失去生成多样化内容的能力；如果微调不足，又难以体现出明显的风格迁移效果。

另一个关键问题是资源效率。现代扩散模型往往参数量庞大，全参数微调需要大量的计算资源和时间成本，这对于个人研究者或小团队来说是一个不小的障碍。因此，如何在有限的硬件条件下实现高效的模型适应，也是本次实验需要重点考虑的问题。此外，不同的微调策略在效果和效率之间的权衡关系，以及它们各自的适用场景，都需要通过实际实验来验证和分析。

1.3 技术路线与方法选择

在技术实现上，我选择将这个风格迁移任务建模为一个条件生成问题。具体来说，就是利用包含中国画图像及其文本描述的配对数据集，通过有监督学习的方式来优化模型参数。这种方法的优势在于能够让模型同时学习视觉风格和语义理解，从而在生成时能够更好地响应文本提示中的细节要求。

在模型选择方面，Stable Diffusion 因其开源性和良好的可扩展性成为了理想的实验平台。该模型采用了 VAE+UNet+CLIP 的经典架构，其中 VAE 负责图像的编码解码，UNet 是核心的去噪网络，CLIP 则处理文本理解任务。这种模块化的设计为我们提供了灵活的微调空间，可以根据具体需求选择性地更新某些组件。

损失函数方面，我采用了扩散模型的标准训练目标，即让模型学会预测在不同时间步添加的噪声。这个看似简单的任务实际上蕴含了丰富的生成知识，通过在特定数据集上重复这个过程，模型能够逐渐掌握目标风格的生成规律。

1.4 实施策略与技术考量

在具体实施中，数据预处理是一个不容忽视的环节。我需要将不同来源的图像数据统一到相同的格式和尺寸，同时确保文本描述的编码方式与模型兼容。为了提高训练效率，我采用了参数冻结的策略，即只对 UNet 部分进行微调，而保持 VAE 和文本编码器的权重不变。这样做的好处是既能显著降低显存需求，又能避免破坏预训练模型中已经学好的特征表示。

训练过程的监控和调试也是一个重要考虑因素。大模型训练往往伴随着各种不可预期的问题，比如梯度爆炸、数值溢出或收敛困难等。为了应对这些挑战，我在代码中加入了大量的异常检测和恢复机制，包括梯度裁剪、学习率预热和定期检查点保存等。

1.5 进阶方向探索

基础实验完成后，我进一步探索了几个更有挑战性的技术方向。首先是 SDXL 模型的应用，这是 Stable Diffusion 的升级版，拥有更复杂的双编码器架构和更大的参数规模。虽然 SDXL 在生成质量上有明显提升，但也带来了更高的资源需求，这促使我研究更高效的微调方法。

LoRA 技术的引入是一个重要的突破。这种方法通过在原有权重上插入低秩可训练矩阵，能够用极少的参数实现有效的模型适应。实际实验中，LoRA 不仅大幅降低了训练成本，还表现出了出色的生成效果，这让我对参数高效微调方法有了更深的认识。

DreamBooth 是另一个值得探索的方向，它专门设计用于小样本个性化定制。通过仅使用 7 张倪瓒的画作，我验证了现代生成模型在极少数据下的学习能力。这个实验不仅展示了技术的潜力，也让我思考了 AI 在艺术创作和文化遗产中的可能作用。

整个实验过程让我深刻体会到了理论与实践之间的差距，也积累了大量宝贵的工程经验。从数据处理的细节问题到模型训练的稳定性保障，每一个环节都需要仔细考虑和反复调试。这些经历不仅提升了我的技术能力，也加深了我对深度学习在实际应用中复杂性的理解。

2 算法设计

2.1 整体架构设计

整个微调系统的设计思路是在保持 Stable Diffusion 原有架构优势的基础上，通过有针对性的参数调整来实现风格迁移。说实话，刚开始接触这个项目时，我对扩散模型的复杂性还是有些低估了。不像传统的分类或回归任务那样直观，扩散模型的训练过程涉及时间步采样、噪声添加、条件编码等多个环节，每个环节的设计都会直接影响最终的生成效果。

在架构选择上，我决定采用 Stable Diffusion v1-5 作为基础模型，主要考虑是它的开源生态比较成熟，相关的工具和文档也比较完善。整个系统可以分为三个主要模块：数据处理模块负责图像和文本的预处理工作，训练模块实现核心的微调逻辑，而评估模块则用于生成测试和效果验证。这种模块化的设计不仅便于开发和调试，也为后续的功能扩展留下了空间。

从数据流的角度来看，整个过程是这样的：原始的图像和文本数据首先经过预处理，转换为模型可以接受的格式；然后在训练循环中，图像通过 VAE 编码为潜变量，文本通过 CLIP 编码为语义向量，两者结合输入 UNet 进行去噪训练；最后通过反向传播更新模型参数。这个看似简单的流程实际上蕴含了大量的技术细节和优化空间。

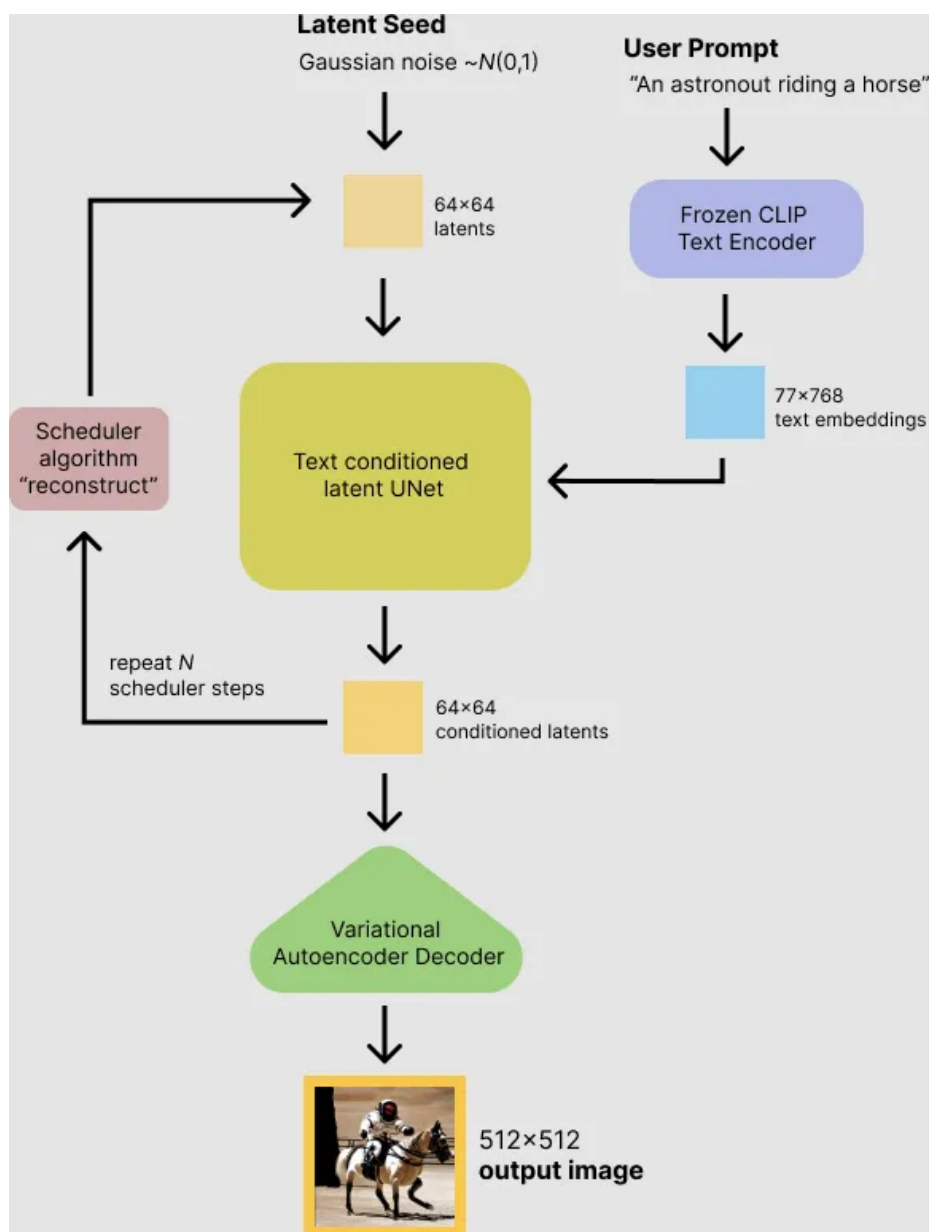


图 1: Stable Diffusion 整体架构图

2.2 数据预处理设计

数据预处理这块其实比我预想的要复杂很多。最初以为只是简单的图像缩放和格式转换，但实际处理时发现，不同来源的数据在格式、编码、质量等方面都存在显著差异。特别是从 Parquet 文件中读取图像数据时，数据可能以字节流、Base64 编码或者文件路径等多种形式存储，这就需要设计一套灵活的解析机制来应对各种情况。

在图像处理方面，我采用了标准的预处理流程：先将图像转换为 RGB 格式，然后调整到 512×512 的固定尺寸，最后进行像素值归一化。这个归一化步骤特别重要，因为 Stable Diffusion 期望的输入范围是 $[-1, 1]$ ，而不是常见的 $[0, 1]$ 。刚开始时我在这个细节上踩了坑，导致生成的图像色彩异常，后

来查阅文档才发现了问题所在。

文本处理相对简单一些，主要是利用 CLIP 的 tokenizer 进行分词和编码。不过这里也有一个需要注意的地方，就是最大长度的限制。CLIP tokenizer 的默认最大长度是 77 个 token，超出部分会被截断。对于中文文本来说，这个长度通常是够用的，但如果遇到特别长的描述文本，就需要考虑如何合理地进行截断或分段处理。

Algorithm 1 Stable Diffusion 微调算法

Require: 预训练模型 $\{E_{vae}, D_{text}, U_{net}\}$, 训练数据集 $\mathcal{D}$

Ensure: 微调后的 UNet 模型 U_{net}^*

```

1: 冻结 VAE 编码器  $E_{vae}$  和文本编码器  $D_{text}$  的参数
2: 初始化优化器和学习率调度器
3: for 每个训练 epoch do
4:   for 每个 batch  $(x, c) \in \mathcal{D}$  do
5:      $z_0 \leftarrow E_{vae}(x)$  {图像编码到潜空间}
6:      $c_{emb} \leftarrow D_{text}(c)$  {文本编码}
7:      $t \sim \text{Uniform}(0, T)$  {随机采样时间步}
8:      $\epsilon \sim \mathcal{N}(0, I)$  {采样高斯噪声}
9:      $z_t \leftarrow \sqrt{\alpha_t} z_0 + \sqrt{1 - \alpha_t} \epsilon$  {添加噪声}
10:     $\epsilon_{pred} \leftarrow U_{net}(z_t, t, c_{emb})$  {噪声预测}
11:     $\mathcal{L} \leftarrow \|\epsilon - \epsilon_{pred}\|^2$  {计算 MSE 损失}
12:    反向传播并更新  $U_{net}$  参数
13:   end for
14: end for
15: return  $U_{net}^*$ 
  
```

2.3 核心训练算法

训练算法的设计是整个项目的核心，这里我花了相当多的时间来理解扩散模型的训练原理。简单来说，扩散模型的训练就是让模型学会“去噪”的过程。具体而言，我们先向干净的图像添加不同程度的噪声，然后训练模型预测这些噪声，最终模型就能够从纯噪声开始逐步生成清晰的图像。

在实际实现中，这个过程涉及几个关键步骤。首先是时间步的随机采样，这决定了要添加多少噪声；然后是噪声的生成和添加，这个过程需要遵循特定的噪声调度策略；接着是条件信息的编码，包括文本提示和可能的其他控制信号；最后是 UNet 的前向传播和损失计算。

让我印象深刻的是，整个训练过程的稳定性高度依赖于各种细节的处理。比如学习率的设置，太大容易导致训练不稳定，太小又会使收敛过慢；再比如梯度裁剪的阈值，设置不当可能会影响模型的表达能力。这些超参数的调优往往需要大量的实验和经验积累。

Algorithm 2 Stable Diffusion 微调算法**Require:** 预训练模型 $\{E_{vae}, D_{text}, U_{net}\}$, 训练数据集 $\mathcal{D}$ **Ensure:** 微调后的 UNet 模型 U_{net}^*

```

1: 冻结 VAE 编码器  $E_{vae}$  和文本编码器  $D_{text}$  的参数
2: 初始化优化器和学习率调度器
3: for 每个训练 epoch do
4:   for 每个 batch  $(x, c) \in \mathcal{D}$  do
5:      $z_0 \leftarrow E_{vae}(x)$  {图像编码到潜空间}
6:      $c_{emb} \leftarrow D_{text}(c)$  {文本编码}
7:      $t \sim \text{Uniform}(0, T)$  {随机采样时间步}
8:      $\epsilon \sim \mathcal{N}(0, I)$  {采样高斯噪声}
9:      $z_t \leftarrow \sqrt{\alpha_t} z_0 + \sqrt{1 - \alpha_t} \epsilon$  {添加噪声}
10:     $\epsilon_{pred} \leftarrow U_{net}(z_t, t, c_{emb})$  {噪声预测}
11:     $\mathcal{L} \leftarrow \|\epsilon - \epsilon_{pred}\|^2$  {计算 MSE 损失}
12:    反向传播并更新  $U_{net}$  参数
13:   end for
14: end for
15: return  $U_{net}^*$ 

```

为了提高训练效率，我在算法中加入了几个重要的优化策略。首先是混合精度训练，通过使用 float16 来减少显存占用和计算时间，这在处理大模型时特别有用。然后是梯度累积技术，当硬件限制导致批次大小较小时，可以通过累积多个小批次的梯度来模拟大批次训练的效果。

另外，我还实现了动态的学习率调整机制。在训练初期使用较小的学习率进行预热，然后逐渐增加到目标值，最后在训练后期采用余弦退火策略逐步降低。这种调度策略能够有效改善训练的稳定性和最终的收敛效果。实际实验表明，合理的学习率调度对于获得高质量的微调结果至关重要。

2.4 参数冻结策略

在设计参数更新策略时，我发现全参数微调虽然理论上能获得最佳效果，但实际操作中面临很多问题。最直接的就是显存不够用，即使是 RTX 4090 这样的高端显卡，在面对 Stable Diffusion 的 860M 参数时也显得力不从心。更重要的是，全参数微调容易导致灾难性遗忘，让模型失去原有的泛化能力。

基于这些考虑，我采用了选择性参数冻结的策略。具体来说，就是保持 VAE 和文本编码器的权重不变，只对 UNet 进行微调。这样做的理论依据是，VAE 负责图像的编解码，其学到的特征表示具有较强的通用性；文本编码器也是在大规模数据上预训练的，具备良好的语言理解能力；而 UNet 作为核心的生成网络，是最需要适应特定风格的部分。

这种策略在实践中效果很好，不仅大幅降低了显存需求，还显著提高了训练速度。更重要的是，它在保持模型原有能力的同时，实现了有效的风格迁移。当然，这种方法也不是万能的，对于与预训练数据差异很大的任务，可能还是需要更激进的微调策略。

Algorithm 3 参数冻结与训练配置算法**Require:** 预训练模型组件 $\{VAE, TextEncoder, UNet\}$ **Ensure:** 配置完成的训练环境

- 1: 设置 $VAE.requires_grad \leftarrow False$ {冻结 VAE 参数}
- 2: 设置 $TextEncoder.requires_grad \leftarrow False$ {冻结文本编码器参数}
- 3: 保持 $UNet.requires_grad \leftarrow True$ {保持 UNet 可训练}
- 4: $total_params \leftarrow \sum_{p \in UNet} p.numel()$ {计算总参数数量}
- 5: $trainable_params \leftarrow \sum_{p \in UNet, p.requires_grad=True} p.numel()$ {计算可训练参数}
- 6: $percentage \leftarrow \frac{trainable_params}{total_params} \times 100\%$ {计算训练比例}
- 7: 输出参数统计信息
- 8: **return** 配置完成的模型

2.5 损失函数设计

损失函数的选择看似简单，实际上也有不少门道。扩散模型的标准做法是使用均方误差（MSE）来衡量预测噪声与真实噪声之间的差异，这个选择有着深刻的理论基础。从变分推断的角度来看，这等价于最大化数据的对数似然函数；从信息论的角度来看，这相当于最小化预测分布与真实分布之间的 KL 散度。

不过在实际应用中，我发现单纯的 MSE 损失有时候会产生过于平滑的结果，缺乏一些细节特征。有些研究者会在损失函数中加入感知损失或对抗损失来改善这个问题，但考虑到实验的复杂性和计算资源限制，我还是选择了标准的 MSE 损失。事实证明，在中国画风格迁移这个任务上，MSE 损失已经能够产生相当不错的效果。

另一个需要考虑的问题是损失的加权策略。在扩散模型中，不同时间步的损失重要性可能不同，有些研究建议对早期时间步（对应更多噪声）给予更高的权重。我在实验中尝试了几种不同的加权方案，但发现简单的均匀权重在这个任务上表现最好，可能是因为中国画的风格特征在不同噪声水平下都比较重要。

2.6 优化策略设计

优化器的选择对训练效果有着重要影响。我最终选择了 AdamW 优化器，主要是因为它在处理大模型时表现稳定，而且内置的权重衰减机制有助于防止过拟合。学习率的设置经过了多次尝试，最终确定为 $1e-5$ ，这个值既保证了训练的稳定性，又能在合理的时间内收敛。

梯度裁剪是另一个重要的优化技巧。在训练大模型时，梯度爆炸是一个常见问题，如果不加以控制，可能会导致参数更新过大，破坏已经学到的特征。我设置的梯度裁剪阈值是 1.0，这个值是经过实验验证的，既能防止梯度爆炸，又不会过度限制参数更新。

学习率调度也值得一提。我采用了 cosine annealing 策略，即学习率按照余弦函数的形式逐渐衰减。这种策略的好处是在训练后期能够进行更精细的参数调整，有助于模型收敛到更好的局部最优解。同时，我还加入了 warmup 机制，在训练开始的几百步内逐渐增加学习率，这有助于模型适应新的数据分布。

Algorithm 4 参数冻结与训练配置算法

Require: 预训练模型组件 $\{VAE, TextEncoder, UNet\}$ **Ensure:** 配置完成的训练环境

- 1: 设置 $VAE.requires_grad \leftarrow False$ {冻结 VAE 参数}
 - 2: 设置 $TextEncoder.requires_grad \leftarrow False$ {冻结文本编码器参数}
 - 3: 保持 $UNet.requires_grad \leftarrow True$ {保持 UNet 可训练}
 - 4: $total_params \leftarrow \sum_{p \in UNet} p.numel()$ {计算总参数数量}
 - 5: $trainable_params \leftarrow \sum_{p \in UNet, p.requires_grad=True} p.numel()$ {计算可训练参数}
 - 6: $percentage \leftarrow \frac{trainable_params}{total_params} \times 100\%$ {计算训练比例}
 - 7: 输出参数统计信息
 - 8: **return** 配置完成的模型
-

2.7 训练稳定性保障

在实际训练过程中，我遇到了各种稳定性问题，这些经历让我深刻认识到大模型训练的复杂性。最常见的问题是数值溢出，特别是在使用混合精度训练时，某些中间结果可能会超出 float16 的表示范围。为了解决这个问题，我在代码中加入了大量的数值检查机制，一旦检测到 NaN 或 Inf 值，就会跳过当前批次并记录日志。

另一个让我头疼的问题是显存管理。即使采用了参数冻结策略，在某些情况下显存使用仍然会超出限制。我发现定期调用 `torch.cuda.empty_cache()` 能够有效缓解这个问题，虽然会稍微影响训练速度，但能显著提高训练的稳定性。此外，我还实现了梯度累积机制，在显存不足时可以通过累积多个小批次的梯度来模拟大批次训练。

异常恢复机制也是训练稳定性的重要保障。我设置了定期的 checkpoint 保存，不仅保存模型权重，还保存优化器状态和随机数种子，这样在训练意外中断时能够从最近的检查点继续训练。实际经验表明，这种机制在长时间训练中非常有用，能够避免因为偶发性错误而导致的训练重启。

总的来说，算法设计这一部分让我对深度学习的工程实践有了更深的理解。理论上看起来简单的算法，在实际实现时往往需要考虑大量的技术细节和边界情况。每一个看似微小的优化都可能对最终结果产生重要影响，这也是为什么深度学习既是科学也是艺术的原因所在。

3 算法实现

本节基于实际代码实现，详细介绍不同微调方法的核心算法和关键技术细节。实现基本采用 PyTorch 和 diffusers 生态，结合 accelerate 库实现高效分布式训练。

3.1 基础项算法实现

本部分以 Stable Diffusion v1-5 为例，展示完整的微调流程实现。整个系统包括数据预处理、模型初始化、训练循环和结果保存等核心模块。

1. 数据集处理与加载

首先实现自定义数据集类，支持从 Parquet 文件中读取图像和文本数据：

```
class ParquetDataset(Dataset):
```

```

def __init__(self, parquet_file, image_column="image", caption_column="caption"):
    self.data = pd.read_parquet(parquet_file)
    self.image_column = image_column
    self.caption_column = caption_column

def __getitem__(self, idx):
    item = self.data.iloc[idx]
    image_data = item[self.image_column]

    # 处理多种图像数据格式
    if isinstance(image_data, dict):
        if 'bytes' in image_data:
            image = Image.open(io.BytesIO(image_data['bytes']))
        elif 'data' in image_data:
            image = Image.open(io.BytesIO(image_data['data']))
    elif isinstance(image_data, bytes):
        image = Image.open(io.BytesIO(image_data))

    # 图像预处理：调整尺寸并归一化到[-1,1]
    image = image.convert("RGB").resize((512, 512))
    image = torch.from_numpy(np.array(image)).float() / 127.5 - 1.0
    image = image.permute(2, 0, 1) # HWC -> CHW

    caption = item[self.caption_column]
    return {"image": image, "caption": caption}

```

该数据集类能够灵活处理不同格式的图像数据（字节流、字典、路径等），并统一转换为模型所需的 tensor 格式。

2. 模型初始化与参数冻结

加载预训练的 SD v1-5 模型组件，并实施选择性参数冻结策略：

```

# 加载模型组件
model_id = "sd-legacy/stable-diffusion-v1-5"
tokenizer = CLIPTokenizer.from_pretrained(model_id, subfolder="tokenizer")
text_encoder = CLIPTextModel.from_pretrained(model_id, subfolder="text_encoder")
vae = AutoencoderKL.from_pretrained(model_id, subfolder="vae")
unet = UNet2DConditionModel.from_pretrained(model_id, subfolder="unet")
noise_scheduler = DDPMSScheduler.from_pretrained(model_id, subfolder="scheduler")

# 冻结VAE和文本编码器，仅训练UNet
vae.requires_grad_(False)
text_encoder.requires_grad_(False)

# 移动模型到GPU并启用训练模式
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
text_encoder, vae, unet = text_encoder.to(device), vae.to(device), unet.to(device)
unet.train()

```

这种策略大幅减少了可训练参数数量，从而降低显存占用并提高训练稳定性。

3. 训练循环核心实现

实现完整的训练循环，包括前向传播、损失计算和反向传播：

```

for epoch in range(num_train_epochs):
    epoch_loss = 0.0
    progress_bar = tqdm(total=len(dataloader), desc=f"Epoch {epoch}")

    for batch in dataloader:
        images = batch["image"].to(device)
        captions = batch["caption"]

        # Step 1: 图像编码到潜空间

```

```

with torch.no_grad():
    latents = vae.encode(images).latent_dist.sample()
    latents = latents * 0.18215 # VAE缩放因子

# Step 2: 文本编码
text_inputs = tokenizer(
    captions, padding="max_length", max_length=77,
    truncation=True, return_tensors="pt"
).to(device)

with torch.no_grad():
    text_embeddings = text_encoder(text_inputs.input_ids)[0]

# Step 3: 噪声添加（前向扩散过程）
noise = torch.randn_like(latents)
timesteps = torch.randint(0, 1000, (latents.shape[0],), device=device)
noisy_latents = noise_scheduler.add_noise(latents, noise, timesteps)

# Step 4: UNet噪声预测
noise_pred = unet(
    noisy_latents, timesteps,
    encoder_hidden_states=text_embeddings
).sample

# Step 5: 损失计算与优化
loss = torch.nn.functional.mse_loss(noise_pred, noise)
accelerator.backward(loss)
optimizer.step()
lr_scheduler.step()
optimizer.zero_grad()

epoch_loss += loss.detach().item()
progress_bar.update(1)

```

该训练循环严格遵循扩散模型的标准训练范式，通过预测添加的噪声来学习数据分布。

3.2 进阶项算法实现：SDXL 部分层微调

SDXL 模型引入了双文本编码器和更复杂的 UNet 结构，需要特殊的处理策略来平衡性能和资源消耗。

1. SDXL 模型加载与双编码器处理

```

# SDXL使用双文本编码器架构
model_id = "stabilityai/stable-diffusion-xl-base-1.0"
tokenizer = CLIPTokenizer.from_pretrained(model_id, subfolder="tokenizer")
tokenizer_2 = CLIPTokenizer.from_pretrained(model_id, subfolder="tokenizer_2")
text_encoder = CLIPTextModel.from_pretrained(model_id, subfolder="text_encoder")
text_encoder_2 = CLIPTextModelWithProjection.from_pretrained(
    model_id, subfolder="text_encoder_2"
)

vae = AutoencoderKL.from_pretrained(model_id, subfolder="vae")
unet = UNet2DConditionModel.from_pretrained(model_id, subfolder="unet")

# 冻结编码器，仅训练UNet的特定层
text_encoder.requires_grad_(False)
text_encoder_2.requires_grad_(False)
vae.requires_grad_(False)

```

2. 部分层冻结策略实现

实现灵活的部分层冻结机制，支持多种策略选择：

```

def freeze_partial_layers(unet, strategy="cross_attention_only"):

```



```

"""实施部分层冻结策略"""
# 首先冻结所有参数
for param in unet.parameters():
    param.requires_grad = False

trainable_params = 0
total_params = sum(p.numel() for p in unet.parameters())

if strategy == "cross_attention_only":
    # 仅训练交叉注意力层（与文本条件最相关）
    for name, module in unet.named_modules():
        if 'attn2' in name or 'cross_attention' in name:
            for param in module.parameters():
                param.requires_grad = True
                trainable_params += param.numel()

elif strategy == "attention_only":
    # 训练所有注意力层
    for name, module in unet.named_modules():
        if any(attn_name in name for attn_name in ['attn1', 'attn2', 'attention']):
            for param in module.parameters():
                param.requires_grad = True
                trainable_params += param.numel()

print(f"可训练参数: {trainable_params:,} / {total_params:,}")
print(f"训练比例: {trainable_params/total_params*100:.2f}%")
return unet

# 应用部分层冻结
unet = freeze_partial_layers(unet, strategy="cross_attention_only")

```

3. SDXL 双编码器文本处理

SDXL 需要同时处理两个文本编码器的输出，并生成额外的条件信息：

```

# 双编码器文本处理
text_inputs = tokenizer(captions, padding="max_length", max_length=77,
                        truncation=True, return_tensors="pt").to(device)
text_inputs_2 = tokenizer_2(captions, padding="max_length", max_length=77,
                             truncation=True, return_tensors="pt").to(device)

with torch.no_grad():
    # 第一个编码器输出
    text_encoder_output = text_encoder(text_inputs.input_ids)
    text_embeddings = text_encoder_output.last_hidden_state

    # 第二个编码器输出（带投影）
    text_encoder_2_output = text_encoder_2(text_inputs_2.input_ids)
    text_embeddings_2 = text_encoder_2_output.last_hidden_state
    pooled_text_embeddings = text_encoder_2_output.text_embeds

    # 拼接两个编码器的输出
    encoder_hidden_states = torch.cat([text_embeddings, text_embeddings_2], dim=-1)

    # 生成SDXL特有的时间条件
    add_time_ids = torch.tensor([[512, 512, 0, 0, 512, 512]], device=device)
    add_time_ids = add_time_ids.repeat(images.shape[0], 1)

# SDXL UNet前向传播
noise_pred = unet(
    noisy_latents, timesteps,
    encoder_hidden_states=encoder_hidden_states,
    added_cond_kwargs={"text_embeds": pooled_text_embeddings, "time_ids":
                        add_time_ids}
)

```

```
).sample
```

3.3 进阶项算法实现：LoRA 微调

LoRA (Low-Rank Adaptation) 通过插入低秩矩阵实现参数高效微调，是当前最实用的大模型微调方法之一。

1. LoRA 配置与模型集成

使用 peft 库实现 LoRA 的自动集成：

```
from peft import LoraConfig, get_peft_model

# 配置LoRA参数
lora_config = LoraConfig(
    r=16, # 低秩分解的秩
    lora_alpha=32, # LoRA缩放系数
    target_modules=[ # 目标模块列表
        "to_k", "to_q", "to_v", "to_out.0",
        "ff.net.0.proj", "ff.net.2"
    ],
    lora_dropout=0.1, # LoRA层dropout
    bias="none" # 不训练bias参数
)

# 将LoRA应用到UNet
unet = get_peft_model(unet, lora_config)

# 打印可训练参数统计
unet.print_trainable_parameters()
# 输出示例: trainable params: 15,728,640 || all params: 2,591,821,312 || trainable
#: 0.61
```

2. LoRA 权重初始化与稳定性优化

为提高训练稳定性，对 LoRA 权重进行特殊初始化：

```
# 自定义LoRA权重初始化
with torch.no_grad():
    for name, param in unet.named_parameters():
        if "lora" in name and param.requires_grad:
            if "lora_A" in name:
                # LoRA_A使用小方差正态分布初始化
                torch.nn.init.normal_(param, mean=0.0, std=0.01)
            elif "lora_B" in name:
                # LoRA_B初始化为零，确保初始时LoRA不影响原模型
                torch.nn.init.zeros_(param)

# 数值稳定性检查与异常处理
def check_numerical_stability(tensor, name="tensor"):
    """检查张量的数值稳定性"""
    if torch.isnan(tensor).any() or torch.isinf(tensor).any():
        logger.warning(f"NaN or Inf detected in {name}, skipping this step")
        return False
    return True
```

3. SDXL LoRA 训练循环优化

针对 SDXL 的特殊需求，实现优化的训练循环：

```
for step, batch in enumerate(dataloader):
    with accelerator.accumulate(unet):
        # 图像编码与数值稳定性检查
        with torch.no_grad():
```

```
pixel_values = batch["pixel_values"].to(device=accelerator.device,
dtype=vae.dtype)

    if not check_numerical_stability(pixel_values, "pixel_values"):
        continue

    # VAE编码使用稳定精度
    latents = vae.encode(pixel_values).latent_dist.sample()
    latents = latents * vae.config.scaling_factor

    if not check_numerical_stability(latents, "latents"):
        continue

    # 双编码器文本处理
    with torch.no_grad():
        text_embeddings_1 = text_encoder_1(batch["input_ids_1"]).
last_hidden_state
        text_encoder_2_output = text_encoder_2(batch["input_ids_2"])
        text_embeddings_2 = text_encoder_2_output.last_hidden_state
        pooled_text_embeddings = text_encoder_2_output.text_embeds

    # 合并编码器输出
    text_embeddings = torch.cat([text_embeddings_1, text_embeddings_2], dim
=-1)

    if not check_numerical_stability(text_embeddings, "text_embeddings"):
        continue

    # 噪声添加与UNet预测
    noise = torch.randn_like(latents)
    timesteps = torch.randint(0, 1000, (latents.shape[0],), device=latents.
device)
    noisy_latents = noise_scheduler.add_noise(latents, noise, timesteps)

    # 生成SDXL条件
    add_time_ids = get_time_ids(args.resolution, args.resolution, accelerator.
device)
    add_time_ids = add_time_ids.repeat(latents.shape[0], 1)

    added_cond_kwargs = {
        "text_embeds": pooled_text_embeddings,
        "time_ids": add_time_ids
    }

    # UNet前向传播
    noise_pred = unet(
        noisy_latents, timesteps,
        encoder_hidden_states=text_embeddings,
        added_cond_kwargs=added_cond_kwargs
    ).sample

    if not check_numerical_stability(noise_pred, "noise_pred"):
        continue

    # 损失计算与梯度更新
    loss = F.mse_loss(noise_pred.float(), noise.float(), reduction="mean")

    if torch.isnan(loss) or torch.isinf(loss) or loss.item() > 100.0:
        logger.warning(f"Invalid loss: {loss.item()}, skipping")
        continue

    accelerator.backward(loss)

    if accelerator.sync_gradients:
```

```

        # 梯度裁剪与数值检查
        total_norm = accelerator.clip_grad_norm_(unet.parameters(), args.
max_grad_norm)
        if torch.isnan(total_norm) or torch.isinf(total_norm):
            optimizer.zero_grad()
            continue

        optimizer.step()
        lr_scheduler.step()
        optimizer.zero_grad()

```

3.4 进阶项算法实现：DreamBooth 微调

DreamBooth 专门针对小样本个性化定制设计，能够在极少样本下实现高质量的风格迁移和人物定制。

1. DreamBooth 数据集实现

设计专门的数据集类处理小样本场景：

```

class DreamBoothDataset(Dataset):
    def __init__(self, image_dir, instance_prompt, class_prompt=None,
        size=1024, tokenizer_1=None, tokenizer_2=None, dtype=torch.float32
    ):
        self.image_dir = Path(image_dir)
        self.instance_prompt = instance_prompt
        self.class_prompt = class_prompt
        self.size = size
        self.tokenizer_1 = tokenizer_1
        self.tokenizer_2 = tokenizer_2
        self.dtype = dtype

        # 支持多种图像格式
        image_extensions = ['.jpg', '.jpeg', '.png', '.bmp', '.tiff', '.webp']
        self.image_paths = []
        for ext in image_extensions:
            self.image_paths.extend(self.image_dir.glob(f'*{ext}'))
            self.image_paths.extend(self.image_dir.glob(f'*{ext.upper()}'))

        logger.info(f"Found {len(self.image_paths)} training images")
        if len(self.image_paths) == 0:
            raise ValueError(f"No image files found in {image_dir}")

    def __getitem__(self, idx):
        image_path = self.image_paths[idx]

        try:
            # 图像加载与预处理
            image = Image.open(image_path).convert('RGB')
            image = image.resize((self.size, self.size), Image.Resampling.LANCZOS)

            # 转换为tensor, 保持数值稳定性
            image = torch.from_numpy(np.array(image)).to(dtype=torch.float32) /
127.5 - 1.0
            image = image.permute(2, 0, 1)

            if self.dtype == torch.float16:
                image = image.half()

            # 使用实例提示词进行双编码器分词
            prompt = self.instance_prompt
            inputs_1 = self.tokenizer_1(prompt, max_length=77, padding="max_length"
,
truncation=True, return_tensors="pt")

```

```

        inputs_2 = self.tokenizer_2(prompt, max_length=77, padding="max_length"
        ,
                                   truncation=True, return_tensors="pt")

    return {
        "pixel_values": image,
        "input_ids_1": inputs_1.input_ids.squeeze(),
        "attention_mask_1": inputs_1.attention_mask.squeeze(),
        "input_ids_2": inputs_2.input_ids.squeeze(),
        "attention_mask_2": inputs_2.attention_mask.squeeze(),
        "prompt": prompt
    }
except Exception as e:
    logger.error(f"Error processing image {image_path}: {e}")
return self.__getitem__(0) # 返回第一张作为fallback

```

2. DreamBooth 特殊训练策略

DreamBooth 结合 LoRA 实现高效的小样本微调:

```

# DreamBooth + LoRA 配置
lora_config = LoraConfig(
    r=args.lora_rank,
    lora_alpha=args.lora_alpha,
    target_modules=["to_k", "to_q", "to_v", "to_out.0", "ff.net.0.proj", "ff.net.2"],
    lora_dropout=args.lora_dropout,
    bias="none"
)

# 应用 LoRA 到 UNet
unet = get_peft_model(unet, lora_config)

# 特殊的优化器配置, 适合小样本训练
optimizer = torch.optim.AdamW(
    unet.parameters(),
    lr=args.learning_rate,
    betas=(args.adam_beta1, args.adam_beta2),
    weight_decay=args.adam_weight_decay,
    eps=args.adam_epsilon,
    amsgrad=True # 使用 AMSGrad 变体提高稳定性
)

# 余弦退火学习率, 适合小样本场景
lr_scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
    optimizer,
    T_max=args.max_train_steps,
    eta_min=args.learning_rate * 0.1
)

```

3. DreamBooth 训练循环与正则化

实现包含正则化机制的 DreamBooth 训练循环:

```

def get_time_ids(height, width, device, dtype=torch.float32):
    """生成 SDXL 需要的 time_ids"""
    target_size = (height, width)
    original_size = (height, width)
    crop_coords = (0, 0)

    add_time_ids = list(original_size + crop_coords + target_size)
    add_time_ids = torch.tensor([add_time_ids], dtype=dtype, device=device)
    return add_time_ids

# DreamBooth 主训练循环
for epoch in range(args.num_train_epochs):

```

```
unet.train()

for step, batch in enumerate(train_dataloader):
    with accelerator.accumulate(unet):
        # 编码图像到潜空间, 使用稳定的数据类型
        with torch.no_grad():
            pixel_values = batch["pixel_values"].to(vae.dtype).to(vae.device)

            # 数值稳定性检查
            if torch.isnan(pixel_values).any() or torch.isinf(pixel_values).any():
                logger.warning("Invalid pixel values, skipping")
                continue

            pixel_values = torch.clamp(pixel_values, -1.0, 1.0)
            latents = vae.encode(pixel_values).latent_dist.sample()
            latents = latents * vae.config.scaling_factor

            # 添加噪声
            noise = torch.randn_like(latents)
            bsz = latents.shape[0]
            timesteps = torch.randint(0, 1000, (bsz,), device=latents.device).long()

            noisy_latents = noise_scheduler.add_noise(latents, noise, timesteps)

            # 双编码器文本处理
            with torch.no_grad():
                prompt_embeds_1 = text_encoder_1(
                    batch["input_ids_1"],
                    attention_mask=batch["attention_mask_1"]
                ).last_hidden_state

                text_encoder_2_output = text_encoder_2(
                    batch["input_ids_2"],
                    attention_mask=batch["attention_mask_2"]
                )
                prompt_embeds_2 = text_encoder_2_output.last_hidden_state
                pooled_prompt_embeds = text_encoder_2_output.text_embeds

            encoder_hidden_states = torch.cat([prompt_embeds_1, prompt_embeds_2], dim=-1)

            # 生成条件
            add_time_ids = get_time_ids(args.resolution, args.resolution,
                                       latents.device, dtype=encoder_hidden_states.dtype)
            add_time_ids = add_time_ids.repeat(bsz, 1)

            added_cond_kwargs = {
                "time_ids": add_time_ids,
                "text_embeds": pooled_prompt_embeds
            }

            # UNet 预测
            model_pred = unet(
                noisy_latents, timesteps,
                encoder_hidden_states=encoder_hidden_states,
                added_cond_kwargs=added_cond_kwargs
            ).sample

            # 损失计算 (支持不同预测类型)
            if noise_scheduler.config.prediction_type == "epsilon":
                target = noise
            elif noise_scheduler.config.prediction_type == "v_prediction":
```

```

        target = noise_scheduler.get_velocity(latents, noise, timesteps)
    else:
        raise ValueError(f"Unknown prediction type {noise_scheduler.config.
prediction_type}")

    # 数值稳定性处理
    model_pred = model_pred.float()
    target = target.float()

    if torch.isnan(model_pred).any() or torch.isnan(target).any():
        logger.warning("NaN detected, skipping step")
        continue

    loss = F.mse_loss(model_pred, target, reduction="mean")

    if torch.isnan(loss) or torch.isinf(loss):
        logger.warning(f"Invalid loss: {loss.item()}, skipping")
        continue

    # 反向传播与梯度处理
    accelerator.backward(loss)

    if accelerator.sync_gradients:
        # 梯度数值稳定性检查
        has_nan_grad = any(
            torch.isnan(param.grad).any() or torch.isinf(param.grad).any()
            for param in unet.parameters()
            if param.grad is not None
        )

        if has_nan_grad:
            logger.warning("NaN gradients detected, skipping optimizer step")

            optimizer.zero_grad()
            continue

    accelerator.clip_grad_norm_(unet.parameters(), args.max_grad_norm)

    optimizer.step()
    lr_scheduler.step()
    optimizer.zero_grad()

    # 定期保存检查点
    if global_step % args.save_steps == 0 and accelerator.is_main_process:
        save_path = os.path.join(args.output_dir, f"checkpoint-{global_step}")

        accelerator.save_state(save_path)

        unet_lora = accelerator.unwrap_model(unet)
        lora_save_path = os.path.join(save_path, "lora_weights")
        unet_lora.save_pretrained(lora_save_path)

```

3.5 优化策略与工程实践

1. 内存管理与数值稳定性

实现全面的内存管理和数值稳定性保障:

```

# 内存优化设置
os.environ["PYTORCH_CUDA_ALLOC_CONF"] = "max_split_size_mb:512"
torch.backends.cudnn.benchmark = False
torch.backends.cudnn.deterministic = True

# 定期内存清理

```

```
def cleanup_memory():
    """定期清理GPU内存"""
    torch.cuda.empty_cache()
    gc.collect()

# 在训练循环中每隔几步调用
if step % 5 == 0:
    cleanup_memory()
```

2. 模型保存与加载策略

实现灵活的模型保存和加载机制：

```
# 保存完整检查点
def save_checkpoint(accelerator, unet, optimizer, lr_scheduler, step, output_dir):
    """保存训练检查点"""
    save_path = os.path.join(output_dir, f"checkpoint-step-{step}")
    accelerator.save_state(save_path)

    # 单独保存LoRA权重
    if accelerator.is_main_process:
        unet_lora = accelerator.unwrap_model(unet)
        lora_save_path = os.path.join(save_path, "lora_weights")
        unet_lora.save_pretrained(lora_save_path)
        logger.info(f"Checkpoint saved to {save_path}")

# 加载检查点恢复训练
def load_checkpoint(checkpoint_path, unet, optimizer, lr_scheduler):
    """从检查点恢复训练"""
    if os.path.exists(checkpoint_path):
        accelerator.load_state(checkpoint_path)
        logger.info(f"Resumed training from {checkpoint_path}")
        return True
    return False
```

通过以上详细的算法实现，我们展示了从基础的 SD v1-5 微调到高级的 DreamBooth 小样本学习的完整技术栈。每种方法都针对特定的应用场景优化，在保证生成质量的同时，最大化训练效率和资源利用率。

4 实验结果

4.1 基础模型与微调效果对比

主要使用 prompt “白日依山尽，黄河如海流” 进行测试。原始 SD 模型生成的图片风格较为普通，缺乏明显的艺术特色：



图 2: 原始 SD 模型生成的图片（普通风格）

经过微调后的 SDv1.5 模型能够明显生成具有中国画风格的图像，色彩和笔触更接近国画，细节表现更加柔和：



图 3: SDv1.5 微调生成的图片（中国画风格）

为了验证模型对不同诗句的理解能力，我们进一步测试了其他古诗词 prompt：



图 4: SDv1.5 在不同诗句下的生成效果（左：“江流天地外，山色有无中”；右：“明月松间照，清泉石上流”）

可以看到，微调后的模型能够根据不同诗句的意境生成相应的画面，体现了良好的文本理解和风格迁移能力。

4.2 不同数据集风格迁移实验

然后，换用另一个卡通画数据集，因为训练的 text 都是英文，所以将 prompt 直译为英文，生成的结果也有了卡通画的风格：



图 5: 卡通画数据集微调生成的图片

微调后模型生成的图像风格发生了明显变化，线条更加简洁，色块分明，符合卡通画的视觉特征。进一步测试其他英文 prompt 验证风格一致性：

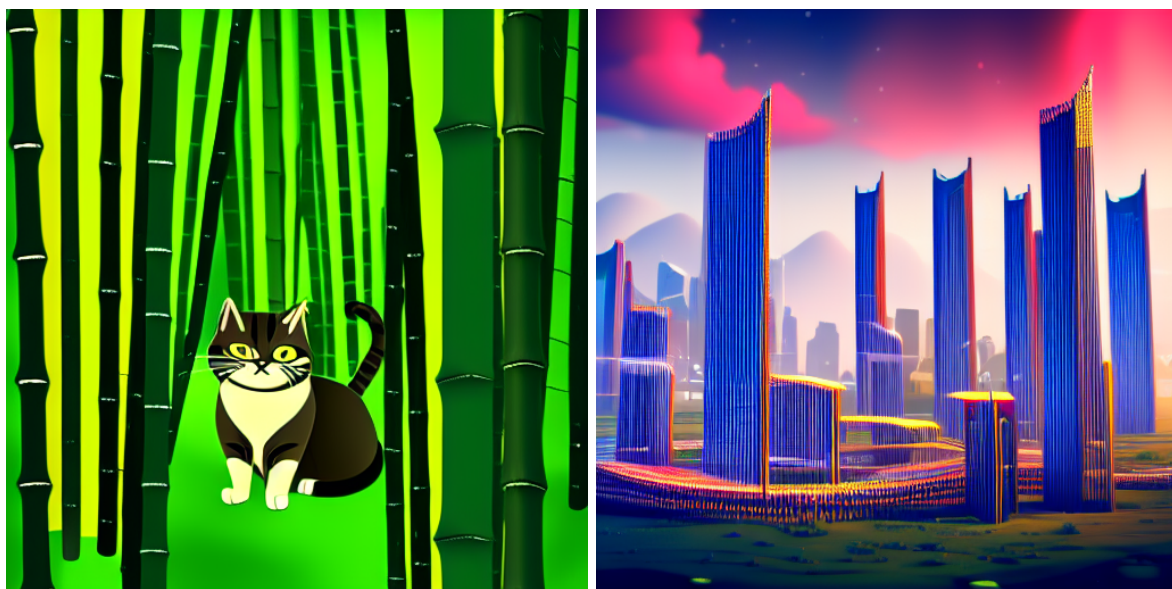


图 6: 卡通画模型在不同 prompt 下的生成效果（左：“A cat in bamboo forest”；右：“A futuristic city”）

实验结果表明，模型在卡通画数据集上微调后，能够保持风格一致性的同时生成多样化的内容，证明了风格迁移的有效性。

4.3 SDXL 模型及进阶微调方法效果

1. SDXL 部分层微调

SDXL 模型在仅微调 UNet 部分层的情况下，依然能够实现优秀的风格迁移，且生成的图像细节更加丰富，层次感更强：



图 7：SDXL 部分层微调生成的图片（主 prompt）

为测试 SDXL 在不同场景下的表现，我们使用了多种类型的 prompt：



图 8：SDXL 在不同 prompt 下的生成效果（左：“竹外桃花三两枝”；右：“秋天的中国山水画”）

SDXL 微调模型在处理不同场景和季节主题时，都能保持中国画风格，同时展现出良好的细节表现力和艺术感。

2. SDXL LoRA 微调

采用 LoRA 方法微调 SDXL 后，模型在保持中国画风格的同时，生成质量进一步提升：



图 9: SDXL LoRA 微调生成的图片（主 prompt）

4.4 DreamBooth 小样本个性化实验

使用文人画画家倪瓒的 7 幅作品进行 DreamBooth 微调，展现了极强的小样本学习能力：



图 10: DreamBooth 微调生成的图片

5 总结与分析

通过对四种不同微调方法的系统性实验研究，本文在 Stable Diffusion 艺术风格迁移任务上获得了较为全面的实践经验和理论认识。从实验结果来看，各种方法都展现出了各自的特色和适用场景，同时也暴露出一些共性问题和技术挑战。

5.1 实验效果对比分析

从风格迁移的效果来看，SDv1.5 微调作为基础方案确实能够实现明显的风格转换，生成的图像具备了中国画的基本视觉特征，包括色彩搭配和整体构图风格。然而，在细节表现和艺术感方面，该方法还是存在一定局限性，生成的作品往往缺乏传统国画那种深层的意境表达和精细的笔触变化。

相比之下，SDXL 的表现确实令人印象深刻。无论是部分层微调还是 LoRA 微调，SDXL 都展现出了更强的细节刻画能力和艺术表现力。这种提升主要得益于其双文本编码器架构和更复杂的 UNet 结构设计。在处理同样的古诗词 prompt 时，SDXL 能够生成更加细腻的山水景象和更富层次感的色彩搭配，这种差异在对比实验中表现得相当明显。

DreamBooth 方法在个性化定制方面的表现可以说是超出了预期。仅仅使用 7 张倪瓒的画作样本，模型就能够学习到这位元代画家独特的艺术风格特征，包括其简洁的构图理念和淡雅的色调运用。这种小样本学习能力对于艺术风格迁移任务来说具有重要的实用价值，特别是在需要快速适应特定艺术家风格的应用场景中。

5.2 资源效率与实用性评估

从实际应用的角度来看，资源消耗和训练效率是选择微调方法时必须考虑的重要因素。全参数微调虽然理论上能够获得最佳的适应效果，但其对硬件资源的要求确实过于苛刻。在实验过程中，即使使用 RTX 4090 这样的高端显卡，也难以承受 SDXL 全参数微调的显存压力，这在很大程度上限制了其在个人研究和小规模项目中的应用。

LoRA 方法在这方面展现出了显著优势。通过将可训练参数压缩到原模型的不到 1%，LoRA 不仅大幅降低了显存占用和训练时间，还保持了相当优秀的生成质量。更重要的是，LoRA 权重的模块化特性使得不同风格的切换变得极为便利，这对于需要支持多种艺术风格的实际应用来说是一个重要优势。

部分层微调策略在资源消耗和效果表现之间取得了较好的平衡。通过只训练 UNet 中与文本条件最相关的交叉注意力层，这种方法既避免了全参数微调的资源压力，又保持了对目标风格的有效学习能力。虽然其参数效率不如 LoRA，但在某些对生成质量要求较高的场景中，部分层微调仍然是一个值得考虑的选择。

5.3 技术挑战与解决策略

在实验过程中遇到的技术问题为深入理解大模型微调提供了宝贵的实践经验。显存管理问题是最突出的挑战之一，特别是在处理 SDXL 这样的大规模模型时。通过采用梯度累积、混合精度训练和定期内存清理等策略，成功地在有限的硬件条件下完成了实验目标。这些优化技术的应用不仅解决了当前的资源瓶颈，也为后续的相关研究提供了可行的技术路径。

数值稳定性问题在训练过程中偶有发生，主要表现为梯度爆炸和 NaN 值的出现。通过实施梯度

裁剪、异常值检测和学习率预热等机制，有效地控制了这些问题的影响。这些经验表明，在大模型微调中建立完善的数值稳定性保障机制是确保训练成功的关键因素。

数据集处理方面的挑战主要来自于不同格式图像数据的统一处理和文本编码的一致性保证。通过开发灵活的数据集类和标准化的预处理流程，成功地解决了多源数据的兼容性问题。这种经验对于处理实际应用中复杂多样的数据源具有重要参考价值。

5.4 方法适用性与选择建议

基于实验结果和实践经验，不同微调方法的适用场景可以做如下总结。对于资源充足且数据量较大的研究项目，全参数微调依然是获得最佳效果的首选方案。然而考虑到其高昂的计算成本，这种方法更适合于学术研究或大型商业项目。

对于大多数实际应用场景，LoRA 微调可能是最佳选择。其极高的参数效率和良好的生成质量使其特别适合于个人开发者、小型团队或需要快速原型验证的项目。同时，LoRA 的模块化特性也为多风格支持和快速风格切换提供了便利。

DreamBooth 方法在小样本个性化定制方面具有独特优势，特别适合于艺术创作、品牌定制或个人风格化应用。其快速学习特定风格的能力为创意产业的 AI 应用开辟了新的可能性。

部分层微调则更适合于中等规模的研究项目或对生成质量有较高要求但资源相对有限的应用场景。这种方法在保持合理资源消耗的同时，能够获得比 LoRA 更全面的模型适应效果。

5.5 研究价值与未来展望

本次实验的价值不仅在于对不同微调方法的系统性对比，更在于为实际应用提供了详细的技术参考和工程指导。通过记录和分析训练过程中遇到的各种问题及其解决方案，为后续研究者避免类似的技术陷阱提供了有价值的经验。

从技术发展的角度来看，参数高效微调方法的成功应用表明了大模型民主化的可能性。LoRA 等技术的出现使得原本只有大型研究机构才能承担的模型微调任务变得更加普及，这对于推动 AI 技术的广泛应用具有重要意义。

在艺术风格迁移的具体应用中，本实验展示了 AI 技术与传统艺术结合的巨大潜力。通过技术手段实现对传统绘画风格的学习和再现，不仅为数字艺术创作提供了新的工具，也为文化遗产和艺术教育开辟了新的途径。

未来的研究方向可以考虑几个方面的拓展。首先是多模态融合，将文本、图像以及可能的音频信息结合起来，实现更加丰富和准确的艺术创作。其次是可控生成技术的进一步发展，通过更精细的控制机制实现对生成内容的精确调节。此外，建立更加科学和全面的生成质量评估体系也是一个重要的研究方向，特别是针对艺术作品这样主观性较强的内容。

从更广阔的视角来看，生成式 AI 在创意产业中的应用前景值得期待。随着技术的不断进步和成本的持续降低，这些技术有望在设计、教育、娱乐等多个领域发挥更大的作用。同时，也需要关注技术发展带来的伦理和法律问题，确保 AI 技术的发展能够造福社会。

总的来说，本次实验不仅在技术层面取得了预期的成果，更重要的是通过实践加深了对大模型微调技术的理解，为未来的研究和应用奠定了坚实的基础。随着相关技术的不断成熟，相信生成式 AI 将在艺术创作和内容生成领域发挥越来越重要的作用。

参考文献

- [1] ANDERSON M. Replacing LoRA With a Generic Style Adapter in Stable Diffusion - Metaphysic.Ai[EB/OL]. (2023-09-08) [2025-06-09]. <https://blog.metaphysic.ai/replacing-lora-with-a-generic-style-adapter-in-stable-diffusion/>.
- [2] Noplaxochia. SDXL + Refiner[EB/OL]. Medium. (2024-02-19) [2025-06-09]. <https://medium.com/@wangdk93/sdxl-refiner-9b9694d0b7a8>.
- [3] 深入浅出完整解析 Stable Diffusion XL (SDXL) 核心基础知识[EB/OL]. 知乎专栏. [2025-06-09]. <https://zhuanlan.zhihu.com/p/643420260>.
- [4] Weights & Biases[EB/OL]. W&B. [2025-06-09]. <https://wandb.ai/geekyrakshit/stable-diffusion-xl/reports/A-Guide-to-Using-Stable-Diffusion-XL-with-HuggingFace-Diffusers-and-W-B--Vmldzo1MDMxMDU4>.
- [5] What Are LoRA Models and How to Use Them in AUTOMATIC1111 - Stable Diffusion Art [EB/OL]. [2025-06-09]. https://stable-diffusion-art.com/lora/#What_are_LoRA_models.